

Pragma: From AI Slot Machine to Governed Software Delivery

September 2026

Most AI development tools operate like slot machines.

A company feeds them prompts, source code, context, and money. The lever gets pulled, and code comes out. Sometimes it works. Sometimes it misses the requirement, breaks a test, ignores an architectural rule, widens the scope, or creates more review work than it saves. Either way, the organization often has little visibility into what happened, why a decision was made, how much it cost, or whether the result followed policy. The first complete accounting may arrive with the monthly AI bill—long after the tokens were spent and the engineering time was lost.

Pragma replaces that gamble with a governed software-delivery system.

Pragma is a collaborative AI coworker that coordinates work from project-management story to reviewed pull request. It does not merely generate code. It manages the process around the code: understanding requirements, gathering missing context, preparing a technical approach, consulting the right people, implementing changes, running the approved build and test plan, opening a pull request, responding to feedback, and recording the final outcome.

The goal is not to remove engineers from software development. It is to give engineering organizations a controlled way to delegate work without surrendering oversight.

Governed workflows, not one-shot prompts

At the center of Pragma is Synapse, a durable workflow engine driven by configurable, version-controlled YAML.

Pragma separates three concepts that conventional coding agents blur together. Triggers decide when work begins. Workflows define reusable units of work. Playbooks coordinate those workflows toward a business outcome through ordered stages, human waits, event handlers, retry policies, and completion rules.

A Story-to-PR playbook can begin when an issue receives an approved label. Pragma verifies that the story has a repository, resolves the responsible team, retrieves the repository's rules, pins the source revision, and prepares the relevant code context. It can then refine the story, propose a technical approach, implement the approved direction, verify the candidate against a frozen build policy, create a branch and commit, open the pull request, notify the team, and wait for review or merge events.

That same operating model extends beyond feature implementation. Pragma can coordinate code review, security review, documentation, modernization assessment, repository analysis, capability mapping, and other repeatable engineering processes. Because the process is represented explicitly, organizations can inspect it, review it, version it, and change it without burying business policy inside an opaque agent prompt.

Human-in-the-loop by design

Human oversight in Pragma is not an emergency stop bolted onto autonomous execution. It is part of the workflow model.

When Pragma needs clarification, encounters a blocker, detects a direction change, exceeds an approved scope, or reaches a required decision point, the execution can suspend safely. The question is routed to the responsible product owner, developer, reviewer, or administrator through the team's working channels. The person's answer is attached to the existing execution, preserved in the record, and used to resume the same workflow with its original context.

A product owner can refine requirements before implementation begins. A developer can correct the technical approach. A reviewer can request changes on the pull request. An authorized person can approve an estimated cost, reject a proposal, take over a conversation, or stop the work entirely. Teams can choose more collaborative operating modes for sensitive work and greater autonomy for mature, well-understood workflows.

Pragma persists waits and checkpoints rather than leaving a worker process running and hoping it survives. External events—such as a reply, pull-request comment, approval, new commit, or merge—are correlated back to the appropriate run. This makes human collaboration durable across restarts, deployments, time zones, and long-running decisions.

The result is controlled autonomy: the machine performs repeatable work, while people retain authority over intent, risk, money, and release decisions.

Governance before generation

Pragma governs what an AI system is allowed to do before it begins editing code.

Repository rules and organizational guidance are loaded from the revision being worked on. The repository, branch, acceptance criteria, source revision, build plan, delivery policy, and responsible team are bound to the job. Baseline verification occurs before the model changes the workspace, and candidate verification occurs afterward. A result that was not actually built or tested is recorded as unverified—not converted into a misleading pass.

Delivery policy determines whether an unverified or partially verified result may proceed. Scope, complexity, story points, and projected cost can trigger an approval before expensive work begins. A human rejection stops the workflow before it consumes additional model calls, pushes a branch, or opens a pull request.

AI providers are interchangeable components inside this governed process. Pragma supports Claude and OpenAI-compatible models, while its canonical adapter model decouples workflows from individual project-management, source-control, notification, documentation, and AI vendors. Integrations include Jira, GitHub, Bitbucket, Microsoft Teams, Slack, and Confluence, with provider-specific details translated into Pragma's common operating model.

The workflow owns the policy. The provider supplies a capability.

Cost and evidence while the work is happening

Pragma turns AI cost from an end-of-month surprise into an operational signal.

AI usage is attributed to the company, team, job, workflow, role, and model responsible for it. Unpriced usage is reported as unknown, never silently treated as zero. Teams can establish approval thresholds so projected work pauses before exceeding an acceptable amount. This connects spending to a particular story and outcome instead of presenting one undifferentiated monthly total.

Engram, Pragma's operating dashboard, makes the system's memory visible. It surfaces jobs, workflow executions, step events, conversations, audit records, configuration, retries, and token usage. Operators can see what is running, what is waiting, what failed, who Pragma needs, and what evidence exists.

Pull requests can carry an evidence section assembled from stored facts rather than model-written claims. That record can include the acceptance contract, files changed, verification status, review findings bound to the delivered commit, cost by AI role, elapsed time, dialogue references, and criteria that were only partially completed or declined. Missing evidence is identified as missing instead of being filled in with optimistic language.

That is the fundamental difference between Pragma and a coding slot machine: every result has a process, a policy, an owner, a cost, and a record.

Security and tenant isolation

Pragma is designed around defense in depth.

Tenant access is scoped through ambient tenant context, application-level query filters, database-backed company membership checks, explicit authorization, tenant-separated workspaces, and PostgreSQL row-level security. Sensitive integration fields use versioned authenticated-encryption envelopes, with per-tenant key authority and fail-closed lifecycle states for suspension, revocation, and deletion. Webhook ingress is authenticated, side effects pass through governed adapters, and significant operations are recorded in the audit trail.

Pragma also treats customer-controlled code as hostile. Hosted build execution is designed around disposable, isolated tasks with no Pragma process, Pragma secret, or credential-bearing task role inside the customer-code boundary. Sandbox controls include privilege removal, syscall restrictions, resource limits, identity switching, and recorded confinement evidence. Hosted execution remains fail-closed when the required boundary cannot be established.

The RLS posture is measurable rather than exaggerated. The current authority manifest catalogs 104 database mappings. Seven high-risk mappings—including AI usage, admissions, and workflow-artifact data—are presently marked as enforced with PostgreSQL FORCE ROW

LEVEL SECURITY; five audit-kernel mappings are explicitly governed by separate controls, and 92 remain planned for database-level enforcement. Those remaining mappings currently depend on application query filters and membership authorization.

Pragma does not describe partial RLS coverage as universal protection. The manifest makes the current boundary and remaining work reviewable.

Built for SOC 2 readiness—without compliance theater

Pragma is being engineered toward SOC 2 readiness as an operating discipline, not a collection of policies assembled immediately before an audit.

The system includes a defined boundary, control ownership, evidence locations, review cadences, access-control procedures, change-management records, incident-response guidance, vendor inventory, data-residency documentation, encryption controls, audit-export protocols, and machine-readable readiness artifacts. Controls are classified separately as implemented, tested, deployed, operating, and live-certified. A checked-in plan or passing local test is never silently promoted into production evidence.

That distinction matters. Pragma is not currently represented as SOC 2 certified, in an active audit, or as having completed an operating-effectiveness period. Production activation and evidence are still required, and known gaps remain in areas such as centralized application-log delivery, database transport security, recovery evidence, key rotation, and enterprise identity capabilities.

For a serious buyer, that honesty is a feature. Pragma is building the machinery required to prove controls continuously while refusing to manufacture confidence that the evidence cannot support.

A system of accountability

Pragma gives engineering leaders a practical path between two unsatisfying choices: keep AI confined to individual developer tools, or grant autonomous agents broad access and hope the output justifies the bill.

With Pragma, organizations can delegate real software-delivery work while keeping workflows explicit, costs attributable, decisions reviewable, data tenant-scoped, human authority intact, and outcomes tied to durable evidence.

Stories go in. Governed, reviewable pull requests come out.

No roulette wheel. No mystery bill. No autonomy without accountability.